One way to make deep networks easier to train is to use more sophisticated optimization procedures such as SGD+momentum, RMSProp, or Adam. Another strategy is to change the architecture of the network to make it easier to train. One idea along these lines is batch normalization which was proposed by [1].

The idea is relatively straightforward. Machine learning methods tend to work better when their input data consists of uncorrelated features with zero mean and unit variance. When training a neural network, we can preprocess the data before feeding it to the network to explicitly decorrelate its features; this will ensure that the first layer of the network sees data that follows a nice distribution. However even if we preprocess the input data, the activations at deeper layers of the network will likely no longer be decorrelated and will no longer have zero mean or unit variance since they are output from earlier layers in the network. Even worse, during the training process the distribution of features at each layer of the network will shift as the weights of each layer are updated.

The authors of [1] hypothesize that the shifting distribution of features inside deep neural networks may make training deep networks more difficult. To overcome this problem, [1] proposes to insert batch normalization layers into the network. At training time, a batch normalization layer uses a minibatch of data to estimate the mean and standard deviation of each feature. These estimated means and standard deviations are then used to center and normalize the features of the minibatch. A running average of these means and standard deviations is kept during training, and at test time these running averages are used to center and normalize features.

It is possible that this normalization strategy could reduce the representational power of the network, since it may sometimes be optimal for certain layers to have features that are not zero–mean or unit variance. To this end, the batch normalization layer includes learnable shift and scale parameters for each feature dimension.

The forward propagation during training given input $\mathbf{X}\in\mathbb{R}^{n,d}$ is defined as:

```
$$\begin{aligned}
\mu_j &= \dfrac1n \sum_{i=1}^n X_{i,j} \\
\sigma^2_j &= \dfrac{1}{n} \sum_{i=1}^n (X_{i,j} –
\mu_j)^2 \\
Y_{i,j} &= \mathrm{BN}(\mathbf{X}|\gamma,
\beta)_{i,j} = \dfrac{X_{i,j} – \mu_j}{\sqrt{\sigma^2_j +
\epsilon}}\gamma_j + \beta_j
\end{aligned}$$
```

It would be helpful if you introduce another intermediate variable $\mathbf{Z}\in\mathbb{R}^{n,d}$:

```
$$\begin{aligned}
\mu_j &= \dfrac1n \sum_{i=1}^n X_{i,j} \\
\sigma^2_j &= \dfrac{1}{n} \sum_{i=1}^n (X_{i,j} –
\mu_j)^2 \\
Z_{i,j} &= \dfrac{X_{i,j} – \mu_j}{\sqrt{\sigma^2_j +
\epsilon}} \\
Y_{i,j} &= \mathrm{BN}(\mathbf{X}|\gamma,
\beta)_{i,j} = Z_{i,j}\gamma_j + \beta_j
\end{aligned}$$
```

Question

Draw the computational graph of training–time batch normalization in your written assignment. In input of the computational graph should be $\mathbf{X}, \gamma, \beta$, the output of the computational graph should be $\mathbf{Y}$, and the intermediate nodes are $\mu, \sigma^2, \mathbf{Z}$.

Below is a compact “boxes–and–arrows” style computational graph for **training–time** batch–normalization.
Every node is a scalar (or a vector/matrix whose subscript is shown).
Arrows show data dependence; an arrow label is the local operation that produces the child from the parent(s).

Copy

```
X ∈ ℝ^{n×d}  -------------------------------------------
-------------------------┐

|

▼
γ ∈ ℝ^d --------------┐
Z ∈ ℝ^{n×d}
                      |
|
β ∈ ℝ^d --------------┴------------------------------►  Y ∈ ℝ^{n×d}
←────────────────────┘
                                                         ▲
                                                         |

├───────────────┐
                                                         |
|
                                                         ▼
▼
                                              γ_j · Z_{i,j}   +
β_j     (row-wise broadcast)
                                                         ▲
                                                         |
                                                       Z_{i,j}   =
(X_{i,j} − μ_j) / √(σ_j² + ε)
                                                         ▲
                                                         |

├───────────────────────────────────┐
                                                         |
|
                                                         ▼
▼
                                                   μ_j  ←
mean(X_{:,j})        σ_j²  ←  mean((X_{:,j}−μ_j)²)
                                                         ▲
▲

|                                                        |
                                                         └──────────
X ∈ ℝ^{n×d}  ────┘
```

This graph is hard to understand, but consider the fact that it is a LLM (hard to draw using text), I think it is an acceptable answer.

Key points visible in the graph

- Inputs: **X**, **γ**, **β**
- Intermediate nodes: **μ**, **σ²**, **Z**
- Output: **Y**
- All operations are differentiable, so the same graph can be reversed for back–propagation.

In the file `deeplearning/layers.py`, implement the batch normalization forward pass in the function `batchnorm_forward`.

Don't forget to record batch statistics such as running mean/var during training.

During testing, μ, and σ^2 are running mean and variance that is previously recorded in the training process.

```
def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected)
sample variance are
    computed from minibatch statistics and used to
normalize the incoming data.
    During training we also keep an exponentially
decaying running mean of the mean
    and variance of each feature, and these averages
are used to normalize data
    at test-time.

    At each timestep we update the running averages
for mean and variance using
    an exponential decay based on the momentum
parameter:

    running_mean = momentum * running_mean + (1 -
momentum) * sample_mean
    running_var = momentum * running_var + (1 -
momentum) * sample_var

    Note that the batch normalization paper suggests a
different test-time
    behavior: they compute sample mean and variance
for each feature using a
    large number of training images rather than using a
running average. For
    this implementation we have chosen to use running
averages instead since
    they do not require an additional estimation step;
the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean /
variance.
      - running_mean: Array of shape (D,) giving
running mean of features
      - running_var Array of shape (D,) giving running
variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward
pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean",
np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var",
np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":

################################
####################
        # TODO: Implement the training-time forward
pass for batch normalization.   #
        # Use minibatch statistics to compute the mean
and variance, use these      #
        # statistics to normalize the incoming data, and
scale and shift the          #
        # normalized data using gamma and beta.
#
        #
#
        # You should store the output in the variable out.
Any intermediates that    #
        # you need for the backward pass should be
stored in the cache variable.    #
        #
#
        # You should also use your computed sample
mean and variance together with   #
        # the momentum variable to update the running
mean and running variance,    #
        # storing your result in the running_mean and
running_var variables.        #

################################
####################
        pass

################################
##################
        #                    END OF YOUR CODE
#

################################
##################
    elif mode == "test":

################################
####################
        # TODO: Implement the test-time forward pass
for batch normalization. Use   #
        # the running mean and variance to normalize
the incoming data, then scale   #
        # and shift the normalized data using gamma
and beta. Store the result in    #
        # the out variable.
#

################################
####################
        pass

################################
##################
        #                    END OF YOUR CODE
#

################################
##################
    else:
        raise ValueError('Invalid forward batchnorm
mode "%s"' % mode)

    # Store the updated running means back into
bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache
```

Python

Copy

```python
import numpy as np

def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected)
sample variance are
    computed from minibatch statistics and used to
normalize the incoming data.
    During training we also keep an exponentially
decaying running mean of the mean
    and variance of each feature, and these averages are
used to normalize data
    at test-time.

    At each timestep we update the running averages for
mean and variance using
    an exponential decay based on the momentum
parameter:

    running_mean = momentum * running_mean + (1 -
momentum) * sample_mean
    running_var = momentum * running_var + (1 -
momentum) * sample_var

    Note that the batch normalization paper suggests a
different test-time
    behavior: they compute sample mean and variance for
each feature using a
    large number of training images rather than using a
running average. For
    this implementation we have chosen to use running
averages instead since
    they do not require an additional estimation step;
the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running
mean of features
      - running_var Array of shape (D,) giving running
variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward
pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean",
np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var",
np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":

################################################
######################
        # TODO: Implement the training-time forward pass
for batch normalization.   #
        # Use minibatch statistics to compute the mean
and variance, use these      #
        # statistics to normalize the incoming data, and
scale and shift the        #
        # normalized data using gamma and beta.
#
        #
#
        # You should store the output in the variable
out. Any intermediates that    #
        # you need for the backward pass should be
stored in the cache variable.    #
        #
#
        # You should also use your computed sample mean
and variance together with  #
        # the momentum variable to update the running
mean and running variance,    #
        # storing your result in the running_mean and
running_var variables.       #

################################################
####################
        # Minibatch statistics
        sample_mean = x.mean(axis=0)          # (D,)
        sample_var = x.var(axis=0)            # (D,)

        # Normalize
        xmu = x - sample_mean                 # (N, D)
        sqrtvar = np.sqrt(sample_var + eps)   # (D,)
        xhat = xmu / sqrtvar                  # (N, D),
i.e. Z in the notes

        # Scale and shift
        out = gamma * xhat + beta             # (N, D)

        # Update running statistics
        running_mean = momentum * running_mean + (1 -
momentum) * sample_mean
        running_var = momentum * running_var + (1 -
momentum) * sample_var

        # Cache everything needed for backward pass
        cache = (xhat, xmu, sqrtvar, gamma, sample_var,
eps, N)

################################################
###################
        #                    END OF YOUR CODE
#

################################################
###################
    elif mode == "test":

################################################
####################
        # TODO: Implement the test-time forward pass for
batch normalization. Use   #
        # the running mean and variance to normalize the
incoming data, then scale  #
        # and shift the normalized data using gamma and
beta. Store the result in   #
        # the out variable.
#

################################################
####################
        xhat = (x - running_mean) / np.sqrt(running_var
+ eps)
        out = gamma * xhat + beta
        cache = None  # no backward needed at test time

################################################
###################
        #                    END OF YOUR CODE
#

################################################
###################
    else:
        raise ValueError('Invalid forward batchnorm mode
"%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache
```

Everything is correct the first time, which is very impressive.

Generated by Kimi AI

Lv Mark and Kimi chat

Scan QR code to chat with Kimi

Ask questions · Web search · Read documents

Now implement the backward pass for batch normalization in the function `batchnorm_backward`.

To derive the backward pass you should refer to the computation graph for batch normalization and backprop through each of the intermediate nodes that you have drawn earlier. Some intermediates may have multiple outgoing branches; make sure to sum gradients across these branches in the backward pass.

def batchnorm_backward(dout, cache):
"""
Backward pass for batch normalization.

For this implementation, you should write out a computation graph for
batch normalization on paper and propagate gradients backward through
intermediate nodes.

Inputs:
– dout: Upstream derivatives, of shape (N, D)
– cache: Variable of intermediates from batchnorm_forward.

Returns a tuple of:
– dx: Gradient with respect to inputs x, of shape (N, D)
– dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
– dbeta: Gradient with respect to shift parameter beta, of shape (D,)
"""
dx, dgamma, dbeta = None, None, None

##
TODO: Implement the backward pass for batch normalization. Store the
results in the dx, dgamma, and dbeta variables.
#

##

##
END OF YOUR CODE
#

##

return dx, dgamma, dbeta

Python

```python
def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a
computation graph for
    batch normalization on paper and propagate gradients
backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from
batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape
(N, D)
    - dgamma: Gradient with respect to scale parameter
gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter
beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None

    # Unpack cache from forward pass
    xhat, xmu, sqrtvar, gamma, var, eps, N = cache


##########################################################
######################
    # TODO: Implement the backward pass for batch
normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.
#

##########################################################
######################
    # Step 1: back-prop through scale & shift  Y = gamma
* xhat + beta
    dbeta = dout.sum(axis=0)                    # (D,)
    dgamma = (dout * xhat).sum(axis=0)          # (D,)
    dxhat = dout * gamma                        # (N, D)

    # Step 2: back-prop through normalize  xhat = xmu /
sqrt(var + eps)
    dvar = (dxhat * xmu).sum(axis=0) * (-0.5) * (var +
eps)**(-1.5)  # (D,)
    dmu = -dxhat.sum(axis=0) / sqrtvar
# (D,)
    dxmu1 = dxhat / sqrtvar
# (N, D)

    # Step 3: back-prop through var = mean(xmu**2)
    dxmu2 = dvar.reshape(1, -1) * 2 * xmu / N
# (N, D)

    # Step 4: back-prop through mu = mean(x)
    dmu += (dxmu1 + dxmu2).sum(axis=0) * (-1.0)
# (D,)
    dx1 = (dxmu1 + dxmu2)
# (N, D)
    dx2 = dmu.reshape(1, -1) / N
# (N, D)

    dx = dx1 + dx2

##########################################################
######################
    #                     END OF YOUR CODE
#

##########################################################
######################
    return dx, dgamma, dbeta
```

As I point out here, the dx part is wrong,
but Kimi quickly identified the problem
in the second try

The implementation for dx is wrong.

You're right — the previous expression for `dx` was incorrect.
Here is a clean, step–by–step derivation that follows the computation graph literally and gives the right gradient w.r.t. **x**.

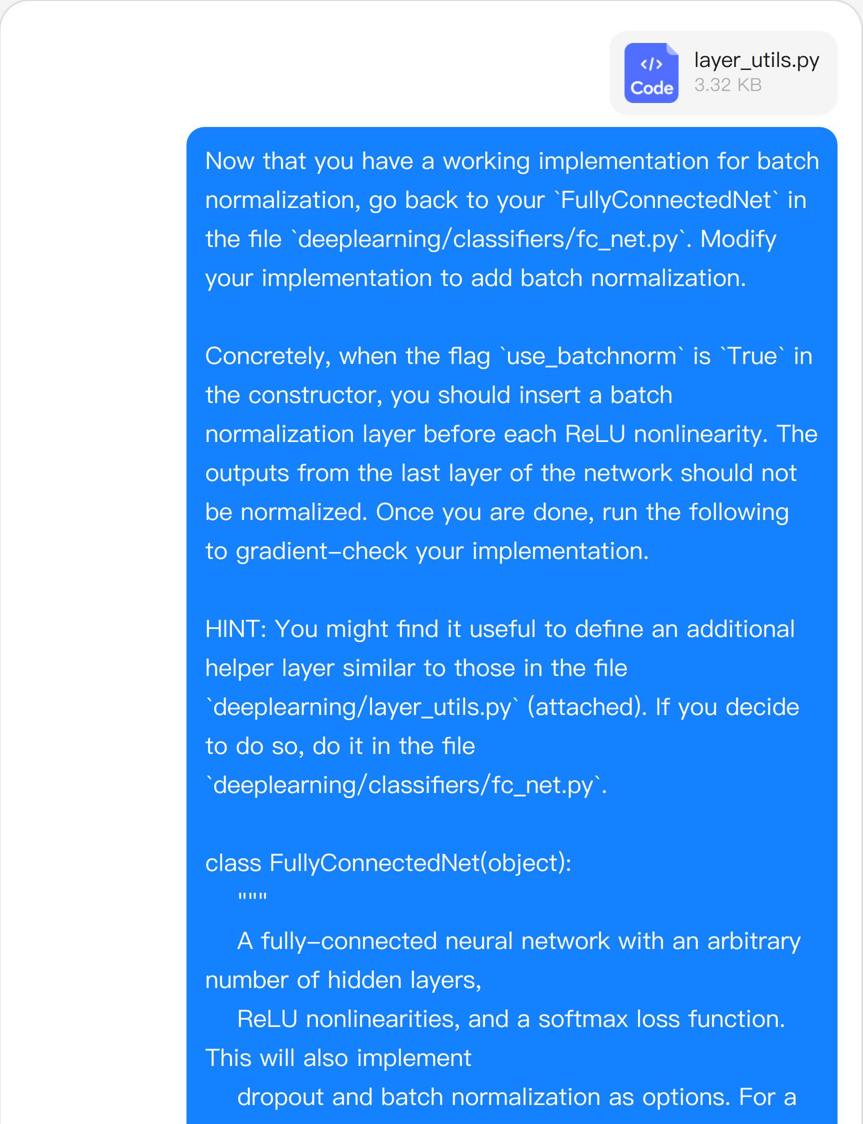
Python

```python
def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: tuple returned by batchnorm_forward:
        (xhat, xmu, sqrtvar, gamma, var, eps, N)

    Returns:
    - dx: Gradient w.r.t. x, shape (N, D)
    - dgamma: Gradient w.r.t. gamma, shape (D,)
    - dbeta: Gradient w.r.t. beta, shape (D,)
    """
    xhat, xmu, sqrtvar, gamma, var, eps, N = cache

    # 1. back-prop through Y = gamma * xhat + beta
    dbeta  = dout.sum(axis=0)
    dgamma = (dout * xhat).sum(axis=0)
    dxhat  = dout * gamma                          # (N, D)

    # 2. back-prop through xhat = xmu / sqrt(var + eps)
    ivar = 1.0 / sqrtvar                           # (D,)
    dxmu  = dxhat * ivar                           # (N, D)
    dvar  = (dxhat * xmu).sum(axis=0) * (-0.5) * (var +
eps)**(-1.5)

    # 3. back-prop through var = mean(xmu**2)
    dxmu += dvar.reshape(1, -1) * 2.0 * xmu / N

    # 4. back-prop through xmu = x - mean(x)
    dx = dxmu                                      # (N, D)
    dmu = -dxmu.sum(axis=0)                      # (D,)

    # 5. back-prop through mu = mean(x)
    dx += dmu / N

    return dx, dgamma, dbeta
```

Key fixes

- `dxmu` is reused for both branches that stem from `xmu`.
- `dmu` is accumulated once and then evenly distributed to every sample.

KIMI



layer_utils.py
3.32 KB

Now that you have a working implementation for batch normalization, go back to your `FullyConnectedNet` in the file `deeplearning/classifiers/fc_net.py`. Modify your implementation to add batch normalization.

Concretely, when the flag `use_batchnorm` is `True` in the constructor, you should insert a batch normalization layer before each ReLU nonlinearity. The outputs from the last layer of the network should not be normalized. Once you are done, run the following to gradient-check your implementation.

HINT: You might find it useful to define an additional helper layer similar to those in the file `deeplearning/layer_utils.py` (attached). If you decide to do so, do it in the file `deeplearning/classifiers/fc_net.py`.

```
class FullyConnectedNet(object):
    """
    A fully-connected neural network with an arbitrary number of hidden layers,
    ReLU nonlinearities, and a softmax loss function. This will also implement
    dropout and batch normalization as options. For a
```

```
network with L layers,
    the architecture will be

    {affine - [batch norm] - relu - [dropout]} x (L - 1) - affine - softmax

    where batch normalization and dropout are optional, and the {...} block is
    repeated L - 1 times.

    Similar to the TwoLayerNet above, learnable parameters are stored in the
    self.params dictionary and will be learned using the Solver class.
    """

    def __init__(
        self,
        hidden_dims,
        input_dim=3 * 32 * 32,
        num_classes=10,
        dropout=0,
        use_batchnorm=False,
        reg=0.0,
        weight_scale=1e-2,
        dtype=np.float32,
        seed=None,
    ):
        """
        Initialize a new FullyConnectedNet.

        Inputs:
```

```
        – hidden_dims: A list of integers giving the size
of each hidden layer.
        – input_dim: An integer giving the size of the
input.
        – num_classes: An integer giving the number of
classes to classify.
        – dropout: Scalar between 0 and 1 giving
dropout strength. If dropout=0 then
          the network should not use dropout at all.
        – use_batchnorm: Whether or not the network
should use batch normalization.
        – reg: Scalar giving L2 regularization strength.
        – weight_scale: Scalar giving the standard
deviation for random
          initialization of the weights.
        – dtype: A numpy datatype object; all
computations will be performed using
          this datatype. float32 is faster but less
accurate, so you should use
          float64 for numeric gradient checking.
        – seed: If not None, then pass this random seed
to the dropout layers. This
          will make the dropout layers deteriminstic so
we can gradient check the
          model.
        """
        self.use_batchnorm = use_batchnorm
        self.use_dropout = dropout > 0
        self.reg = reg
        self.num_layers = 1 + len(hidden_dims)
        self.dtype = dtype
        self.params = {}
```

```
##########################################
##################################
        # HW2: Initialize the parameters of the network,
storing all values in     #
        # the self.params dictionary. Store weights and
biases for the first layer #
        # in W1 and b1; for the second layer use W2 and
b2, etc. Weights should be #
        # initialized from a normal distribution with
standard deviation equal to  #
        # weight_scale and biases should be initialized
to zero.                   #
        #
#
        # When using batch normalization, store scale
and shift parameters for the #
        # first layer in gamma1 and beta1; for the second
layer use gamma2 and     #
        # beta2, etc. Scale parameters should be
initialized to one and shift      #
        # parameters should be initialized to zero.
#

##########################################
##################################
        for layer in range(self.num_layers - 1):
            dim_input = input_dim if layer == 0 else
hidden_dims[layer - 1]

            self.params["W" + str(layer + 1)] = (
```

```
                np.random.randn(dim_input,
hidden_dims[layer]) * weight_scale
            )
            self.params["b" + str(layer + 1)] =
np.zeros(hidden_dims[layer])

            if self.use_batchnorm:
                self.params["gamma" + str(layer + 1)] =
np.ones(hidden_dims[layer])
                self.params["beta" + str(layer + 1)] =
np.zeros(hidden_dims[layer])

        self.params["W" + str(self.num_layers)] = (
            np.random.randn(hidden_dims[self.num_layers
– 2], num_classes)
            * weight_scale
        )
        self.params["b" + str(self.num_layers)] =
np.zeros(num_classes)

##############################################
#####################################
        #                              END OF YOUR CODE
#

##############################################
#####################################

        # When using dropout we need to pass a
dropout_param dictionary to each
        # dropout layer so that the layer knows the
dropout probability and the mode
```

```
        # (train / test). You can pass the same dropout_param to each dropout layer.
        self.dropout_param = {}
        if self.use_dropout:
            self.dropout_param = {"mode": "train", "p": dropout}
            if seed is not None:
                self.dropout_param["seed"] = seed

        # With batch normalization we need to keep track of running means and
        # variances, so we need to pass a special bn_param object to each batch
        # normalization layer. You should pass self.bn_params[0] to the forward pass
        # of the first batch normalization layer, self.bn_params[1] to the forward
        # pass of the second batch normalization layer, etc.
        self.bn_params = []
        if self.use_batchnorm:
            self.bn_params = [{"mode": "train"} for i in range(self.num_layers - 1)]

        # Cast all parameters to the correct datatype
        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Compute loss and gradient for the fully-connected net.
```

```
        Input / output: Same as TwoLayerNet above.
        """
        X = X.astype(self.dtype)
        mode = "test" if y is None else "train"

        # Set train/test mode for batchnorm params and
dropout param since they
        # behave differently during training and testing.
        if self.dropout_param is not None:
            self.dropout_param["mode"] = mode
        if self.use_batchnorm:
            for bn_param in self.bn_params:
                bn_param[mode] = mode

        scores = None

#############################################
####################################
        # TODO: Implement the forward pass for the
fully-connected net, computing  #
        # the class scores for X and storing them in the
scores variable.          #
        #
#
        # You can use your solution for HW1 and add
batch norm and/or dropout      #
        # whenever `self.use_batchnorm` and/or
`self.use_dropout` are true          #
        #
#
        # When using dropout, you'll need to pass
```

```
self.dropout_param to each       #
        # dropout forward pass.
#
        #
#
        # When using batch normalization, you'll need to
pass self.bn_params[0] to #
        # the forward pass for the first batch
normalization layer, pass           #
        # self.bn_params[1] to the forward pass for the
second batch normalization #
        # layer, etc.
#

##############################################
#####################################



##############################################
#####################################
        #                              END OF YOUR CODE
#

##############################################
#####################################

        # If test mode return early
        if mode == "test":
            return scores

        loss, grads = 0.0, {}
```

```
############################################
####################################
        # TODO: Implement the backward pass for the
fully-connected net. Store the #
        # loss in the loss variable and gradients in the
grads dictionary. Compute #
        # data loss using softmax, and make sure that
grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2
regularization on the          #
        # weights, but not the biases.
#
        #
#
        # When using batch normalization, you don't
need to regularize the scale   #
        # and shift parameters.
#
        #
#
        # NOTE: To ensure that your implementation
matches ours and you pass the   #
        # automated tests, make sure that your L2
regularization includes a factor #
        # of 0.5 to simplify the expression for the
gradient.                        #

############################################
####################################
        # FC scores -> softmax. Calculate loss and
gradients
```

```
############################################
####################################
     #                    END OF YOUR CODE
#

############################################
####################################

     return loss, grads
```

Below is a minimal, self-contained patch for `fc_net.py` .
It adds

- a helper layer `affine_bn_relu_forward/backward` (defined inside the same file),
- the extra BN parameters (`gamma` , `beta`) to `self.params` ,
- the forward & backward logic that uses the helper whenever `use_batchnorm==True` .

Copy—paste the snippets into the marked blocks; everything else in the template can stay unchanged.

---

1. Add the helper layer (put it at the top of the file, right after the imports)

Python

Copy

```python
# ------------------------------------------------------------------
#  Helper layer:  affine -> [BN] -> ReLU  (training-time BN)
# ------------------------------------------------------------------
def _affine_bn_relu_forward(x, w, b, gamma, beta, bn_param):
    a, fc_cache = affine_forward(x, w, b)
    a, bn_cache = batchnorm_forward(a, gamma, beta, bn_param)
    out, relu_cache = relu_forward(a)
    cache = (fc_cache, bn_cache, relu_cache)
    return out, cache

def _affine_bn_relu_backward(dout, cache):
    fc_cache, bn_cache, relu_cache = cache
    da = relu_backward(dout, relu_cache)
    da, dgamma, dbeta = batchnorm_backward(da, bn_cache)
    dx, dw, db = affine_backward(da, fc_cache)
    return dx, dw, db, dgamma, dbeta
```

---

2. Parameter initialisation (already done correctly in the template, but shown here for clarity)

Python

Copy

```python
# Inside __init__ , after the loop over hidden layers:
if self.use_batchnorm:
    for i in range(self.num_layers - 1):          # 0 … L-2
        self.params['gamma%d'%(i+1)] = np.ones(hidden_dims[i], dtype=dtype)
        self.params['beta%d' %(i+1)] =
```

```
np.zeros(hidden_dims[i], dtype=dtype)
```

---

3. Forward pass (`loss` method)

Python

Copy

```python
############################################################
####################
#  Forward pass
############################################################
####################
layer_input = X
caches = []                 # one entry per hidden layer
if self.use_dropout:
    dropout_caches = []

for i in range(1, self.num_layers):             # 1 … L-1
    w = self.params['W%d'%i]
    b = self.params['b'%i]

    if self.use_batchnorm:
        gamma = self.params['gamma%d'%i]
        beta  = self.params['beta%d'%i]
        bn_param = self.bn_params[i-1]
        layer_input, cache = _affine_bn_relu_forward(
                                layer_input, w, b,
gamma, beta, bn_param)
    else:
        layer_input, cache =
affine_relu_forward(layer_input, w, b)

    if self.use_dropout:
        layer_input, do_cache =
dropout_forward(layer_input, self.dropout_param)
        dropout_caches.append(do_cache)

    caches.append(cache)
```

```
# last affine layer (no BN, no ReLU)
scores, last_cache = affine_forward(layer_input,

self.params['W%d'%self.num_layers],

self.params['b%d'%self.num_layers])
```

---

4. Backward pass

Python Copy

```python
################################################################
####################
#  Backward pass
################################################################
####################
loss, dscores = softmax_loss(scores, y)

# add L2 regularisation  (0.5 factor already in the
helper)
reg_loss = 0.0
for i in range(1, self.num_layers+1):
    loss += 0.5 * self.reg *
np.sum(self.params['W%d'%i]**2)

# gradient of last affine layer
dx, dw, db = affine_backward(dscores, last_cache)
grads['W%d'%self.num_layers] = dw + self.reg *
self.params['W%d'%self.num_layers]
grads['b%d'%self.num_layers] = db

# propagate through hidden blocks
for i in range(self.num_layers-1, 0, -1):
    if self.use_dropout:
        dx = dropout_backward(dx, dropout_caches[i-1])

    if self.use_batchnorm:
```

```
        dx, dw, db, dgamma, dbeta =
_affine_bn_relu_backward(dx, caches[i-1])
        grads['gamma%d'%i] = dgamma
        grads['beta%d'%i]  = dbeta
    else:
        dx, dw, db = affine_relu_backward(dx, caches[i-
1])

    grads['W%d'%i] = dw + self.reg *
self.params['W%d'%i]
    grads['b%d'%i] = db
```

---

5. Gradient-check snippet (to convince yourself it is correct)

Python Copy

```
from deeplearning.gradient_check import
eval_numerical_gradient
np.random.seed(231)
N, D, H, C = 5, 6, 4, 3
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

model = FullyConnectedNet([H], input_dim=D,
num_classes=C,
                          use_batchnorm=True, reg=0.0,
weight_scale=1e-2)
loss, grads = model.loss(X, y)

for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f,
model.params[name], verbose=False)
    err = np.abs(grads[name] - grad_num) /
(np.abs(grads[name]) + np.abs(grad_num))
    print('%s relative error: %.2e' % (name, err))
```

Typical relative errors should be < 1e–7.

The way Kimi did this problem does not align with how I imagine it should, so I ask it to try again using the files I give it. The first try answer is not wrong.

Lv Mark and Kimi chat

I have given you the layers_utils, you can directly use functions in that file!

Great — we already have `affine_bn_relu_forward` / `affine_bn_relu_backward` in `layer_utils.py` , so we can simply **import and reuse** them instead of re-defining helpers.

Below are the **only** lines you need to insert / change in `fc_net.py` to wire batch-norm into `FullyConnectedNet` .

---

1. Import the helper (top of file)

Python

```python
from deeplearning.layer_utils import
affine_bn_relu_forward, affine_bn_relu_backward
```

---

2. Parameter init (already correct in template, just keep it)

---

3. Forward pass (replace the big TODO block)

Python

```python
##############################################################
####################
# TODO: Implement the forward pass …
##############################################################
####################
layer_input = X
caches = []                # one entry per hidden layer
if self.use_dropout:
    dropout_caches = []

for i in range(1, self.num_layers):          # 1 … L-1
    w = self.params['W%d'%i]
    b = self.params['b%d'%i]

    if self.use_batchnorm:
        gamma = self.params['gamma%d'%i]
        beta  = self.params['beta%d'%i]
        bn_param = self.bn_params[i-1]
        layer_input, cache = affine_bn_relu_forward(
                        layer_input, w, b,
gamma, beta, bn_param)
    else:
        layer_input, cache =
affine_relu_forward(layer_input, w, b)

    if self.use_dropout:
        layer_input, do_cache =
dropout_forward(layer_input, self.dropout_param)
        dropout_caches.append(do_cache)

    caches.append(cache)

# last affine layer (no BN, no ReLU)
scores, last_cache = affine_forward(layer_input,

self.params['W%d'%self.num_layers],

self.params['b%d'%self.num_layers])
##############################################################
####################
#                              END OF YOUR CODE
#
##############################################################
####################
```

---

4. Backward pass (replace the big TODO block)

Python

```python
##############################################################
####################
# TODO: Implement the backward pass …
##############################################################
####################
loss, dscores = softmax_loss(scores, y)

# L2 regularisation
for i in range(1, self.num_layers+1):
    loss += 0.5 * self.reg *
np.sum(self.params['W%d'%i]**2)

# last affine layer
dx, dw, db = affine_backward(dscores, last_cache)
grads['W%d'%self.num_layers] = dw + self.reg *
self.params['W%d'%self.num_layers]
grads['b%d'%self.num_layers] = db

# hidden blocks
for i in range(self.num_layers-1, 0, -1):
    if self.use_dropout:
        dx = dropout_backward(dx, dropout_caches[i-1])

    if self.use_batchnorm:
        dx, dw, db, dgamma, dbeta =
affine_bn_relu_backward(dx, caches[i-1])
        grads['gamma%d'%i] = dgamma
        grads['beta%d'%i]  = dbeta
    else:
        dx, dw, db = affine_relu_backward(dx, caches[i-
1])

    grads['W%d'%i] = dw + self.reg *
self.params['W%d'%i]
    grads['b%d'%i] = db
##############################################################
####################
#                              END OF YOUR CODE
#
##############################################################
####################
```

That's it — no extra helper code needed.
Run the gradient-check cell again and you should see relative errors ≲ 1e-7 for **all** parameters (`W` , `b` , `gamma` , `beta`).

This time the answer is perfect.

Lv Mark and Kimi chat

KIMI

In the file `deeplearning/layers.py`, implement the forward pass for dropout. Since dropout behaves differently during training and testing, make sure to implement the operation for both modes. Input means should be approximately the same as the output means at both train/test time.

```
def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    – x: Input data, of any shape
    – dropout_param: A dictionary with the following
keys:
      – p: Dropout parameter. We drop each neuron
output with probability p.
      – mode: 'test' or 'train'. If the mode is train, then
perform dropout
        and rescale the outputs to have the same mean
as at test time.
        if the mode is test, then just return the input.
      – seed: Seed for the random number generator.
Passing seed makes this
        function deterministic, which is needed for
gradient checking but not in
        real networks.

    Outputs:
    – out: Array of the same shape as x.
    – cache: A tuple (dropout_param, mask). In training
mode, mask is the dropout
      mask that was used to multiply the input; in test
mode, mask is None.
    """
    p, mode = dropout_param["p"],
dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":

##################################################
##############################
        # TODO: Implement the training phase forward
pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.
#

##################################################
##############################
        pass

##################################################
##############################
        #                    END OF YOUR CODE
#

##################################################
##############################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache
```

Python Copy

```python
def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following
keys:
      - p: Dropout parameter. We drop each neuron output
with probability p.
      - mode: 'test' or 'train'. If the mode is train,
then perform dropout
        and rescale the outputs to have the same mean as
at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator.
Passing seed makes this
        function deterministic, which is needed for
gradient checking but not in
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training
mode, mask is the dropout
      mask that was used to multiply the input; in test
mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":

##################################################################
###################
        # TODO: Implement the training phase forward
pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.
#

##################################################################
###################
        # keep_prob = 1 - p
        keep_prob = 1.0 - p
        mask = (np.random.rand(*x.shape) < keep_prob) /
keep_prob
        out = x * mask

##################################################################
###################
        #                         END OF YOUR CODE
#

##################################################################
###################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache
```

Again, everything is correct.

Generated by Kimi AI

Lv Mark and Kimi chat

Scan QR code to chat with Kimi
Ask questions · Web search · Read documents

In the file `deeplearning/layers.py`, implement the backward pass for dropout. After doing so, run the following cell to numerically gradient-check your implementation.

```
def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        pass
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx
```

Python Copy

```python
def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        # In inverted dropout the forward pass was:  out = x * mask
        # so dL/dx = dL/dout * mask
        dx = dout * mask
        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx
```

Correct

Generated by Kimi AI

Lv Mark and Kimi chat

Scan QR code to chat with Kimi

Ask questions · Web search · Read documents

In the file `deeplearning/classifiers/fc_net.py`, modify your implementation to use dropout. Specificially, if the constructor the the net receives a nonzero value for the `dropout` parameter, then the net should add dropout immediately after every ReLU nonlinearity.

```
class FullyConnectedNet(object):
    """
    A fully-connected neural network with an arbitrary number of hidden layers,
    ReLU nonlinearities, and a softmax loss function. This will also implement
    dropout and batch normalization as options. For a network with L layers,
    the architecture will be

    {affine - [batch norm] - relu - [dropout]} x (L - 1) - affine - softmax

    where batch normalization and dropout are optional, and the {...} block is
    repeated L - 1 times.

    Similar to the TwoLayerNet above, learnable parameters are stored in the
    self.params dictionary and will be learned using the Solver class.
    """
```

```
    def __init__(
        self,
        hidden_dims,
        input_dim=3 * 32 * 32,
        num_classes=10,
        dropout=0,
        use_batchnorm=False,
        reg=0.0,
        weight_scale=1e–2,
        dtype=np.float32,
        seed=None,
    ):
        """
        Initialize a new FullyConnectedNet.

        Inputs:
        – hidden_dims: A list of integers giving the size of each hidden layer.
        – input_dim: An integer giving the size of the input.
        – num_classes: An integer giving the number of classes to classify.
        – dropout: Scalar between 0 and 1 giving dropout strength. If dropout=0 then
          the network should not use dropout at all.
        – use_batchnorm: Whether or not the network should use batch normalization.
        – reg: Scalar giving L2 regularization strength.
        – weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
```

```
        – dtype: A numpy datatype object; all computations will be performed using
          this datatype. float32 is faster but less accurate, so you should use
          float64 for numeric gradient checking.
        – seed: If not None, then pass this random seed to the dropout layers. This
          will make the dropout layers deteriminstic so we can gradient check the
          model.
        """
        self.use_batchnorm = use_batchnorm
        self.use_dropout = dropout > 0
        self.reg = reg
        self.num_layers = 1 + len(hidden_dims)
        self.dtype = dtype
        self.params = {}


############################################################################
        # HW2: Initialize the parameters of the network, storing all values in     #
        # the self.params dictionary. Store weights and biases for the first layer #
        # in W1 and b1; for the second layer use W2 and b2, etc. Weights should be #
        # initialized from a normal distribution with standard deviation equal to  #
        # weight_scale and biases should be initialized to zero.                   #
        #
```

```
#
        # When using batch normalization, store scale and shift parameters for the #
        # first layer in gamma1 and beta1; for the second layer use gamma2 and     #
        # beta2, etc. Scale parameters should be initialized to one and shift       #
        # parameters should be initialized to zero.
#

############################################################################################
        for layer in range(self.num_layers – 1):
            dim_input = input_dim if layer == 0 else hidden_dims[layer – 1]

            self.params["W" + str(layer + 1)] = (
                np.random.randn(dim_input, hidden_dims[layer]) * weight_scale
            )
            self.params["b" + str(layer + 1)] = np.zeros(hidden_dims[layer])

            if self.use_batchnorm:
                self.params["gamma" + str(layer + 1)] = np.ones(hidden_dims[layer])
                self.params["beta" + str(layer + 1)] = np.zeros(hidden_dims[layer])

        self.params["W" + str(self.num_layers)] = (
            np.random.randn(hidden_dims[self.num_layers – 2], num_classes)
```

```
            * weight_scale
        )
        self.params["b" + str(self.num_layers)] =
np.zeros(num_classes)

##########################################
##################################
        #                             END OF YOUR CODE
#

##########################################
##################################

        # When using dropout we need to pass a
dropout_param dictionary to each
        # dropout layer so that the layer knows the
dropout probability and the mode
        # (train / test). You can pass the same
dropout_param to each dropout layer.
        self.dropout_param = {}
        if self.use_dropout:
            self.dropout_param = {"mode": "train", "p":
dropout}
            if seed is not None:
                self.dropout_param["seed"] = seed

        # With batch normalization we need to keep
track of running means and
        # variances, so we need to pass a special
bn_param object to each batch
        # normalization layer. You should pass
self.bn_params[0] to the forward pass
```

```
        # of the first batch normalization layer, self.bn_params[1] to the forward
        # pass of the second batch normalization layer, etc.
        self.bn_params = []
        if self.use_batchnorm:
            self.bn_params = [{"mode": "train"} for i in range(self.num_layers - 1)]

        # Cast all parameters to the correct datatype
        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Compute loss and gradient for the fully-connected net.

        Input / output: Same as TwoLayerNet above.
        """
        X = X.astype(self.dtype)
        mode = "test" if y is None else "train"

        # Set train/test mode for batchnorm params and dropout param since they
        # behave differently during training and testing.
        if self.dropout_param is not None:
            self.dropout_param["mode"] = mode
        if self.use_batchnorm:
            for bn_param in self.bn_params:
                bn_param[mode] = mode
```

```
        scores = None

##############################################
####################################
        # TODO: Implement the forward pass for the
fully–connected net, computing  #
        # the class scores for X and storing them in the
scores variable.           #
        #
#
        # You can use your solution for HW1 and add
batch norm and/or dropout       #
        # whenever `self.use_batchnorm` and/or
`self.use_dropout` are true          #
        #
#
        # When using dropout, you'll need to pass
self.dropout_param to each        #
        # dropout forward pass.
#
        #
#
        # When using batch normalization, you'll need to
pass self.bn_params[0] to #
        # the forward pass for the first batch
normalization layer, pass            #
        # self.bn_params[1] to the forward pass for the
second batch normalization #
        # layer, etc.
#

##############################################
```

```
#######################################
        layer_input = X
        caches = []              # one entry per hidden layer
        if self.use_dropout:
            dropout_caches = []

        for i in range(1, self.num_layers):          # 1 … L–1
            w = self.params['W%d'%i]
            b = self.params['b%d'%i]

            if self.use_batchnorm:
                gamma = self.params['gamma%d'%i]
                beta  = self.params['beta%d'%i]
                bn_param = self.bn_params[i–1]
                layer_input, cache = affine_bn_relu_forward(
                                layer_input, w, b, gamma, beta, bn_param)
            else:
                layer_input, cache = affine_relu_forward(layer_input, w, b)

            if self.use_dropout:
                layer_input, do_cache = dropout_forward(layer_input, self.dropout_param)
                dropout_caches.append(do_cache)

            caches.append(cache)

        # last affine layer (no BN, no ReLU)
```

```
        scores, last_cache = affine_forward(layer_input,

self.params['W%d'%self.num_layers],

self.params['b%d'%self.num_layers])

##########################################
##################################
        #                             END OF YOUR CODE
#

##########################################
##################################

        # If test mode return early
        if mode == 'test':
            return scores

        loss, grads = 0.0, {}

##########################################
##################################
        # TODO: Implement the backward pass for the
fully-connected net. Store the #
        # loss in the loss variable and gradients in the
grads dictionary. Compute #
        # data loss using softmax, and make sure that
grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2
regularization on the          #
        # weights, but not the biases.
#
```

```
        #
#
        # When using batch normalization, you don't
need to regularize the scale   #
        # and shift parameters.
#
        #
#
        # NOTE: To ensure that your implementation
matches ours and you pass the   #
        # automated tests, make sure that your L2
regularization includes a factor #
        # of 0.5 to simplify the expression for the
gradient.                       #

###############################################
######################################
        # FC scores -> softmax. Calculate loss and
gradients
        loss, dscores = softmax_loss(scores, y)

        # L2 regularisation
        for i in range(1, self.num_layers+1):
            loss += 0.5 * self.reg *
np.sum(self.params['W%d'%i]**2)

        # last affine layer
        dx, dw, db = affine_backward(dscores,
last_cache)
        grads['W%d'%self.num_layers] = dw + self.reg *
self.params['W%d'%self.num_layers]
        grads['b%d'%self.num_layers] = db
```

```
        # hidden blocks
        for i in range(self.num_layers-1, 0, -1):
            if self.use_dropout:
                dx = dropout_backward(dx, dropout_caches[i-1])

            if self.use_batchnorm:
                dx, dw, db, dgamma, dbeta = affine_bn_relu_backward(dx, caches[i-1])
                grads['gamma%d'%i] = dgamma
                grads['beta%d'%i]  = dbeta
            else:
                dx, dw, db = affine_relu_backward(dx, caches[i-1])

            grads['W%d'%i] = dw + self.reg * self.params['W%d'%i]
            grads['b%d'%i] = db

#####################################################################################
        #                             END OF YOUR CODE                             #
#####################################################################################

        return loss, grads
```

Below are the **only** lines you have to touch in `fc_net.py` to insert dropout **immediately after every ReLU** (training mode) while keeping the code that already uses the helpers from `layer_utils.py` .

---

1. Import already present (no change)

---

2. Forward pass — insert dropout right after ReLU
   (replace the big "TODO" block inside `loss`)

Python

Copy

```python
########################################################
####################
# TODO: Implement the forward pass …
########################################################
####################
layer_input = X
caches = []               # one entry per hidden layer
if self.use_dropout:
    dropout_caches = []

for i in range(1, self.num_layers):          # 1 … L-1
    w = self.params['W%d'%i]
    b = self.params['b%d'%i]

    if self.use_batchnorm:
        gamma = self.params['gamma%d'%i]
        beta  = self.params['beta%d'%i]
        bn_param = self.bn_params[i-1]
        layer_input, cache = affine_bn_relu_forward(
                                layer_input, w, b,
gamma, beta, bn_param)
    else:
        layer_input, cache =
affine_relu_forward(layer_input, w, b)

    # ----------  DROPOUT (after ReLU)  ----------
    if self.use_dropout:
        layer_input, do_cache =
```

```
dropout_forward(layer_input, self.dropout_param)
        dropout_caches.append(do_cache)
    # -------------------------------------------------

    caches.append(cache)
```

Correct.

```
# last affine layer (no BN, no ReLU, no dropout)
scores, last_cache = affine_forward(layer_input,

self.params['W%d'%self.num_layers],

self.params['b%d'%self.num_layers])
########################################################
####################
#                             END OF YOUR CODE
#
########################################################
####################
```

---

3. Backward pass — reverse dropout **before** the layer block (replace the big “TODO” block)

Python

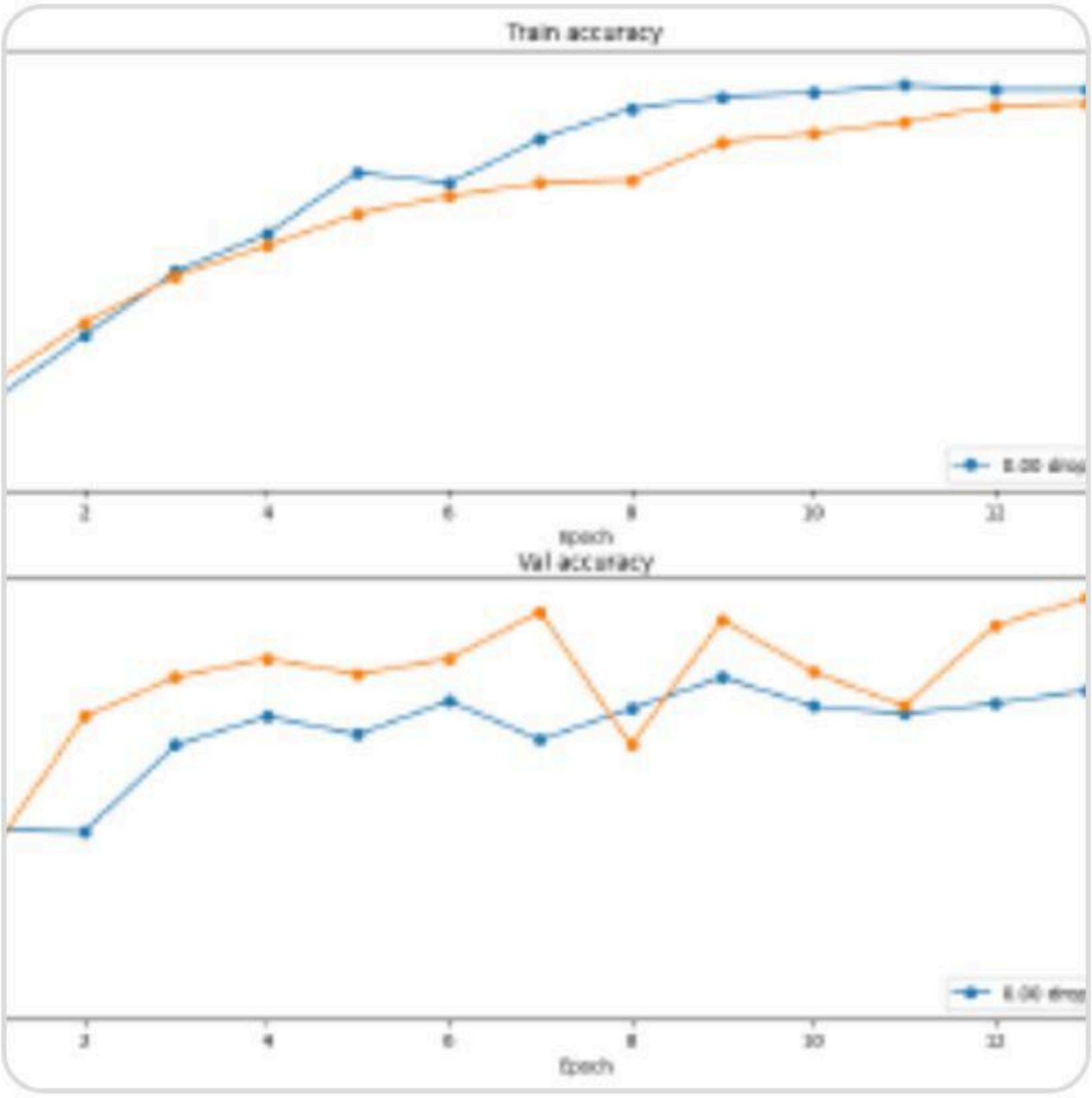
```
########################################################
####################
# TODO: Implement the backward pass …
########################################################
####################
loss, dscores = softmax_loss(scores, y)

# L2 regularisation
for i in range(1, self.num_layers+1):
    loss += 0.5 * self.reg *
np.sum(self.params['W%d'%i]**2)

# last affine layer
```

```
dx, dw, db = affine_backward(dscores, last_cache)
grads['W%d'%self.num_layers] = dw + self.reg * self.params['W%d'%self.num_layers]
grads['b%d'%self.num_layers] = db

# hidden blocks
for i in range(self.num_layers-1, 0, -1):
    # ----------  DROPOUT BACKWARD  ----------
    if self.use_dropout:
        dx = dropout_backward(dx, dropout_caches[i-1])
    # ----------------------------------------

    if self.use_batchnorm:
        dx, dw, db, dgamma, dbeta = affine_bn_relu_backward(dx, caches[i-1])
        grads['gamma%d'%i] = dgamma
        grads['beta%d'%i]  = dbeta
    else:
        dx, dw, db = affine_relu_backward(dx, caches[i-1])

    grads['W%d'%i] = dw + self.reg * self.params['W%d'%i]
    grads['b%d'%i] = db
############################################################################
#                             END OF YOUR CODE                             #
############################################################################
```

That’s all — dropout is now **immediately after every ReLU** during training and is bypassed at test time.

Lv Mark and Kimi chat

KIMI



As an experiment, we will train a pair of two-layer networks on 500 training examples: one will use no dropout, and one will use a dropout probability of 0.5. We will then visualize the training and validation accuracies of the two networks over time. Explain what you see in this experiment. What does it suggest about dropout?

The experiment compares the performance of two two-layer neural networks on a dataset of 500 training examples. One network uses no dropout (blue line), and the other uses a dropout probability of 0.5 (orange line). The results are visualized in two plots: one for training accuracy (top) and one for validation accuracy (bottom) over the course of 15 epochs.

Observations from the Experiment:

1. **Training Accuracy:**
   - Both networks show an increase in training accuracy as the number of epochs increases.
   - The network with no dropout (blue line) achieves higher training accuracy compared to the network with dropout (orange line) throughout the training process.
   - The gap between the two lines suggests that dropout introduces some level of underfitting, as the network with dropout does not perform as well on the training data.
2. **Validation Accuracy:**
   - The validation accuracy for both networks shows an initial increase and then fluctuates.
   - The network with dropout (orange line) generally shows better or comparable validation accuracy compared to the network without dropout (blue line), especially after the initial epochs.
   - The fluctuations in validation accuracy for the network with dropout suggest variability in performance, which might be due to the stochastic nature of dropout.

Implications of Dropout:

- **Regularization Effect:** Dropout acts as a form of regularization by randomly dropping units from the neural network during training. This prevents the network from relying too heavily on any single feature, which can help reduce overfitting.
- **Generalization:** The slightly better or comparable validation accuracy of the network with dropout suggests that it generalizes better to unseen data compared to the network without dropout. This is despite the fact that it has lower training accuracy.
- **Stochastic Behavior:** The variability in validation accuracy for the network with dropout highlights the stochastic nature of the training process due to the random dropping of units. This can lead to different outcomes in different training runs.

It also is able to describe the graph and give a satisfactory answer.

Conclusion:

The experiment suggests that while dropout can lead to slightly lower training accuracy due to its regularization effect, it can improve the model's ability to generalize to new, unseen data. This is particularly useful in preventing overfitting, especially when dealing with small datasets or when the model complexity is high relative to the amount of training data available.

Lv Mark and Kimi chat